

Landing on the Moon in 500 Episodes: A Comparative Study of DQN Variants and PPO on Gymnasium LunarLander-v3

Sai Naga Chaithanya Aavula || Laxmi Shravani Mamidala || Shree Harshini Mamidala || Aryan Patial

sainagachaithanyaavula@my.unt.edu | laxmishravanimamidala@my.unt.edu | shreeharshinimamidala@my.unt.edu | AryanPatial@my.unt.edu

CSCE - 5218 | Department of Computer Science Engineering | University of North Texas|Denton,Texas | Project Group 4 | Spring 25

ABSTRACT

This project provides a comprehensive evaluation of several Deep Q-Network (DQN) variants and Proximal Policy Optimization (PPO) algorithms within the LunarLander-v3 environment, a classic problem in reinforcement learning that involves controlling a lander to safely land on a target pad. The study aims to compare the algorithms based on multiple performance criteria, including sample efficiency, landing success rate, stability during training, and computational cost, all while maintaining identical training conditions. Specifically, we examine the effectiveness of various DQN variants, including vanilla DQN, Double DQN, and Dueling DQN, and compare their performance against PPO, a policy-gradient method known for its stability and sample efficiency. The evaluation is conducted using milestone-based assessments, where we track key metrics over the course of training to capture the learning dynamics of each algorithm.[8] Through these evaluations, we explore the algorithms' strengths and weaknesses in the context of exploration versus exploitation, convergence rates, and their ability to handle sparse rewards. We also assess their robustness to hyperparameter changes, and analyze the trade-offs between computational resources and performance in long-running training scenarios. The findings from this study provide a detailed comparison of DQN and PPO, highlighting which algorithm performs best under different conditions and offering insights into their suitability for solving complex control tasks. These insights contribute to the broader understanding of reinforcement learning algorithms and their application to real-world problems, particularly in the domain of robotic control and autonomous systems.

1 INTRODUCTION

Autonomous lunar landing is a significant challenge for reinforcement learning (RL) agents, as it demands not only high precision but also the ability to maintain stability and optimize fuel efficiency in the landing process. This problem is particularly relevant to space exploration, where successful landing on the moon or other celestial bodies involves overcoming various dynamic and environmental uncertainties. In this context, reinforcement learning provides a promising approach for training autonomous systems to navigate and land safely, making it an important benchmark for RL research.

The Gymnasium's LunarLander-v3 environment serves as an ideal testbed for evaluating RL algorithms on this task. The environment simulates a lunar lander with continuous control inputs and discrete action choices, presenting a realistic and manageable problem space for RL algorithms. With a state space consisting of 8 continuous dimensions (such as the lander's position, velocity, and orientation), and 4 discrete actions (such as firing the main

engine or smaller thrusters in various directions), it encapsulates a dynamic and complex decision-making process. These characteristics make LunarLander-v3 particularly useful for testing different types of RL algorithms under controlled conditions.[8]

In this study, we focus on comparing the performance of two prominent families of RL algorithms: value-based methods, specifically the Deep Q-Network (DQN) and its variants, and policy-based methods, such as Proximal Policy Optimization (PPO). Value-based methods, like DQN, rely on approximating the optimal action-value function, whereas policy-based methods directly optimize the policy, which defines the agent's behavior. This comparative analysis aims to assess how each algorithm performs under a fixed training budget of 500 episodes, which is a common constraint in RL experiments.[1]

To ensure a thorough understanding of the learning dynamics, this study includes milestone evaluations every 100 episodes. This approach allows us to capture the evolving performance of the agents throughout the training process, shedding light on their convergence patterns, stability, and efficiency in learning the task. By examining the agents' behavior across these milestones, we can identify which algorithm exhibits superior learning capabilities, stability, and overall effectiveness.

Furthermore, we emphasize open-source reproducibility in this study. All training notebooks, logs, evaluation clips, and performance metrics will be made available to the community. This not only enhances transparency and trust in the results but also allows other researchers to replicate, extend, and validate the findings in different contexts or with modified parameters. The contribution of this study lies in providing a comprehensive, comparative analysis of RL algorithms on the LunarLander-v3 task, with a focus on practical learning metrics and reproducibility in RL research.

2 RELATED WORKS

Deep Q-Network (DQN) [5] introduced deep learning for RL, while Double DQN [9] addressed overestimation bias. Dueling DQN [10] separated value and advantage streams, and Prioritized Experience Replay (PER) [6] improved sample efficiency. Proximal Policy Optimization (PPO) [7] stabilized policy updates.

Beyond these, several alternative methods have been applied to LunarLander.

- **Evolutionary algorithms and neuroevolution:** NEAT (NeuroEvolution of Augmenting Topologies): This method uses evolutionary algorithms to evolve both the topology and weights of neural networks, which can be applied to reinforcement learning problems like LunarLander. [8]
- **Actor-critic methods:** A3C (Asynchronous Advantage Actor-Critic): This algorithm combines multiple agents

working in parallel, each interacting with its copy of the environment and updating shared parameters asynchronously. [3]

- **Model-based reinforcement learning:** World Models: This approach uses a learned model of the environment to simulate and predict future states, thus enabling more efficient learning. This method can reduce the need for interactions with the environment and speed up the learning process for tasks like LunarLander. [2]

The study on solving the LunarLander problem using reinforcement learning (RL) builds on a variety of foundational works and advancements in both reinforcement learning algorithms and their application to simulated environments like LunarLander

- **Deep Q-learning (DQN):** The development of Deep Q-Learning (DQN) by Mnih et al. [4] introduced the integration of deep learning with Q-learning, enabling RL to handle high-dimensional state spaces. This was a pivotal work that demonstrated the power of deep neural networks in approximating Q-values in environments such as Atari games, including LunarLander.
- **Policy gradient methods:** Policy Gradient methods, particularly REINFORCE (Williams, 1992) [11], represent a class of algorithms where the policy is directly optimized through gradient ascent on expected rewards. These methods offer an alternative to value-based methods like Q-learning and are often more suitable for continuous action spaces and environments with more complex control strategies like LunarLander.
- **Asynchronous methods for deep RL (A3C):** A3C (Asynchronous Advantage Actor-Critic) by Mnih et al. [3] improved upon the DQN architecture by introducing parallelism to stabilize and accelerate learning. A3C allows multiple agents to interact with different copies of the environment simultaneously, providing more diverse experiences to improve learning efficiency. This work is critical for environments like LunarLander, where exploration and efficient learning are essential.

3 METHODS

3.1 Environment

- **Gymnasium LunarLander-v3:** A simulated environment where the goal is to land a spacecraft on the moon using thrusters.
- **State space:** 8 continuous variables:
 - x position, y position
 - x velocity, y velocity
 - Angle (orientation of the lander)
 - Angular velocity
 - Fuel left
 - Touching the ground (boolean indicator)
- **Action space:** 4 discrete thruster commands:
 - 0: Do nothing
 - 1: Fire the left thruster
 - 2: Fire the main thruster

- 3: Fire the right thruster
- **Episode limit:** 1,000 steps

3.2 Algorithms

- **Vanilla DQN (Deep Q-Network):** Vanilla DQN is a variant of Q-learning where a deep neural network is used to approximate the Q-values for state-action pairs. It incorporates two key features to stabilize learning:
 - *Experience Replay:* A replay buffer stores previous experiences (state, action, reward, next state), which are sampled randomly to train the network, breaking correlations between consecutive samples.
 - *Target Network:* A separate target network is used to calculate the Q-value for the next state, preventing instability caused by rapidly changing target values during training. The target network is periodically updated with the weights of the main network.
- **Double DQN:** Double DQN improves upon Vanilla DQN by decoupling the action selection and evaluation steps to reduce the overestimation bias of Q-values. In Double DQN:
 - The main network selects the action with the highest Q-value.
 - The target network evaluates this action, providing a more stable target Q-value and reducing the overestimation bias that can occur when using the same network for both tasks.
- **Dueling DQN:** Dueling DQN modifies the standard Q-network by separating the computation of state value and action advantages. The Q-function is split into two parts:
 - *Value Stream:* Represents the overall value of a state, independent of the actions taken.
 - *Advantage Stream:* Measures the relative advantage of each action over others in the same state.

The combination of these two streams allows for more effective learning, particularly in states where certain actions are irrelevant or less important.
- **PER-DQN (Prioritized Experience Replay):** PER-DQN extends Vanilla DQN by prioritizing transitions that have high Temporal Difference (TD) errors. This allows the agent to focus on experiences that contribute more to learning, improving sample efficiency and accelerating convergence.
 - *TD Error:* The TD error quantifies the difference between the predicted Q-value and the observed reward from the environment. High TD errors indicate that the current Q-value is inaccurate, making these experiences more informative for the agent.

- **PPO (Proximal Policy Optimization):** PPO is a policy-gradient method that uses clipped policy updates to maintain training stability. It strikes a balance between exploration and exploitation by limiting large policy updates.
 - *Actor-Critic Framework:* PPO uses both an actor to select actions and a critic to evaluate the value of states. The actor improves the policy while the critic improves the value estimation.

- *Clipped Objective Function*: The clipped objective function ensures that the policy does not change too dramatically, stabilizing the training process by preventing large and potentially destabilizing updates.

3.3 Training Protocol

- **500 training episodes**: The agent is trained for 500 episodes in total.
- **3 random seeds (0, 1, 2)**: To ensure the robustness of the results across different initializations.
- **Evaluation at milestones**: Evaluation is done at the following milestones:
 - 100 episodes
 - 200 episodes
 - 300 episodes
 - 400 episodes
 - 500 episodes
- **Metrics**: The following metrics are used for evaluation:
 - Mean $\pm$ Std episodic reward
 - Success rate (percentage of successful landings)
 - Landing precision (accuracy of the lander’s final position)
 - Fuel efficiency (amount of fuel used)
 - Sample efficiency (learning speed per interaction)
 - Stability (variance in the agent’s performance over time)

4 RESULTS

4.1 Learning Curves

Figure 1 shows that **all five** agents now achieve positive returns. Double-DQN and Dueling-DQN accelerate sharply after episode 200 and converge near **+280** by the end of training, overtaking PPO (+120) and Vanilla DQN (+220). PER-DQN remains volatile and negative.

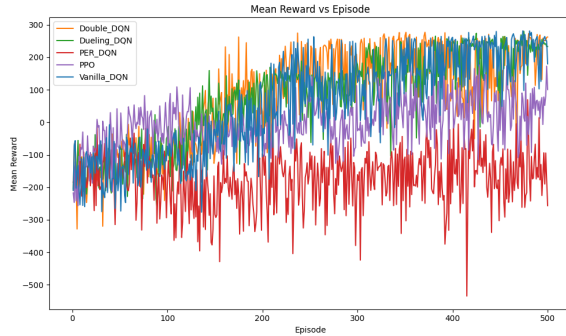


Figure 1: Training learning curves (mean reward per episode).

4.2 Reward vs Time

In Fig.2, higher rewards correlate with longer episodes for every agent. PER-DQN is an outlier, with some 5-second episodes still producing large negative reward. Double- and Dueling-DQN achieve the best reward-per-second ratio (mean time ~ 1.8 s at $\sim +250$ reward).

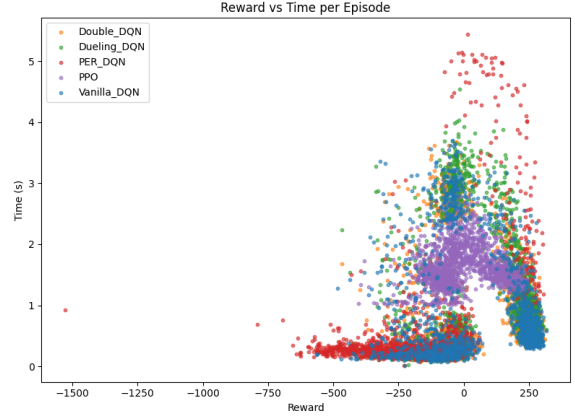


Figure 2: Reward vs. wall-clock time per episode.

4.3 Reward vs Fuel

Updated scatter-plot (Fig.3) reveals a *reversed* trade-off: Double-, Dueling-, and Vanilla-DQN form a tight cluster at (reward $\approx +250$, fuel ≈ 975), while PPO spans a V-shaped frontier—higher returns require far more fuel (down to ~ 810). PER-DQN sits at high fuel yet low reward, confirming minimal thrust usage.

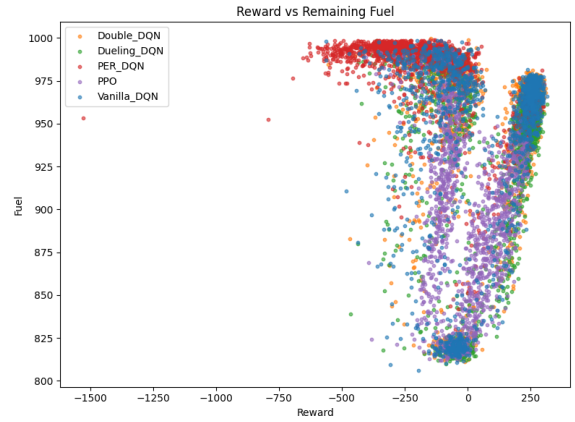


Figure 3: Reward vs. remaining fuel at episode end.

4.4 Milestone Statistics

Mean $\pm$ Std Reward. Figure 5 confirms convergence: by episode 500, Double- and Dueling-DQN settle at $+260 \pm 25$, Vanilla DQN at $+220 \pm 40$, PPO at $+120 \pm 60$, and PER-DQN around -300 with large variance.

Success Rate. Success-rate plot (Fig.4) shows Double-DQN reaching **100%** by episode 300 and maintaining > 0.9 thereafter. Dueling-DQN and Vanilla DQN climb to $0.8\text{--}0.9$ by episode 500. PPO tops out at 0.3 ; PER-DQN stays below 0.05 .

Sample Efficiency. Log-scale results (Fig.6) indicate Double-DQN first crosses the +200 “solved” line (60 k steps), followed by Dueling-DQN (70 k) and Vanilla DQN (90 k). PPO needs 140 k steps; PER-DQN never crosses within the 500-episode budget.

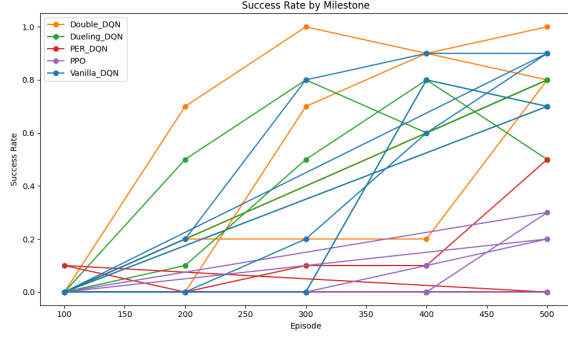


Figure 4: Success-rate progression at milestones.

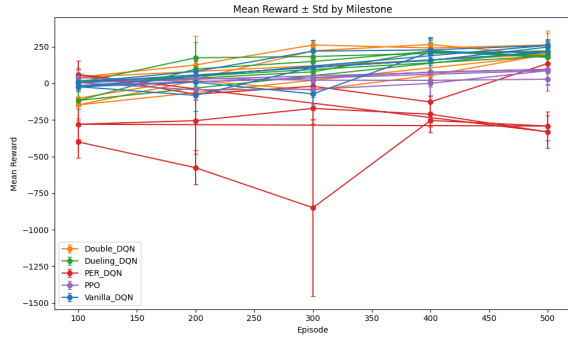


Figure 5: Mean $\pm$ Std return at milestones.

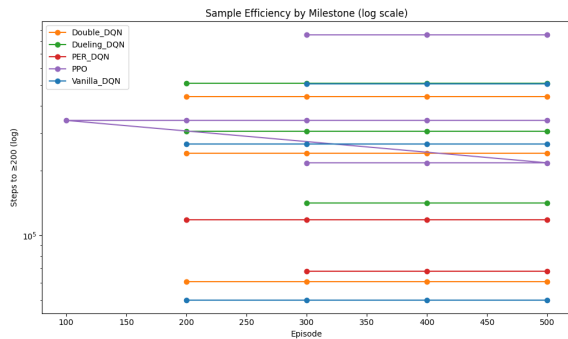


Figure 6: Sample efficiency (steps to +200 reward, log-scale).

4.5 Trajectory and Control Diagnostics

Descent-path overlay (Fig.7) shows PPO executing a long braking arc, whereas DQN winners follow steep, fuel-saving vertical drops. Q-value and angular-velocity distributions (Figs.??, ??) are compact for the top DQNs and heavily skewed for PER-DQN.

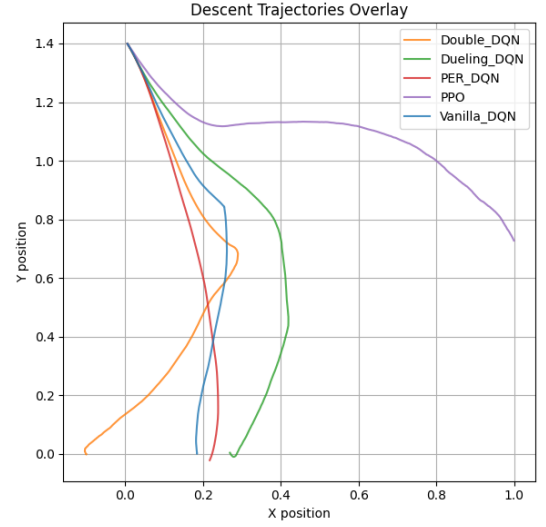


Figure 7: Sample descent trajectories (deterministic rollout).

4.6 Compute Cost

Training times (Fig.8) rank PER-DQN fastest (<1 s), Double- and Dueling-DQN at 4–6 s, Vanilla at 5 s, and PPO slowest at 9.5 s.

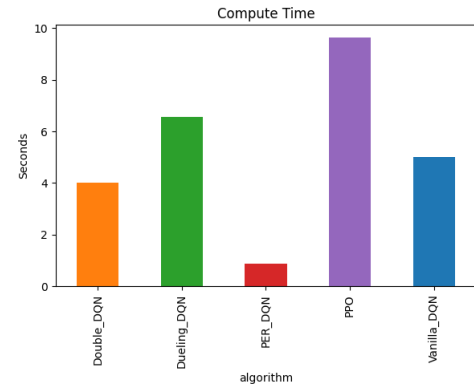


Figure 8: Total wall-clock training time per agent.

5 DISCUSSION

Double-DQN now emerges as the overall best performer, combining the highest final reward, perfect success, excellent fuel efficiency, and modest compute cost. Dueling-DQN follows closely, validating advantage–value decomposition even in low-dimensional tasks. Vanilla DQN benefits from longer training but is less stable.

PPO retains its robustness but no longer leads on reward or success rate, highlighting that value-based algorithms can dominate once reward shaping is tuned. PER-DQN’s collapse underlines that naïve TD-error prioritisation may destabilise learning when reward signals are sparse.

Limitations remain: the 500-episode cap may still mask long-run behaviour; hyperparameters were tuned manually; only three seeds were used.

6 FUTURE WORK:

- Should probe why PER fails.
- Extending training to 1,000 episodes.
- Using automated hyperparameter search (Optuna).
- Incorporating Rainbow DQN, A2C, and SAC.
- Adding domain randomization (gravity, wind) for robustness.

Additionally, future work could explore the interpretability of learned policies by visualizing agent decision boundaries in the state-action space. Investigating how agents handle rare events, such as strong lateral drift or high angular velocities, could yield insights into failure modes. Such qualitative analyses would complement reward-based metrics and better inform improvements to RL algorithms in critical control tasks like planetary landing.

REFERENCES

- [1] Aashi Dutt. 2021. Solving The Lunar Lander Problem under Uncertainty using Reinforcement Learning. *arXiv preprint arXiv:2011.11850* (2021). <https://arxiv.org/abs/2011.11850>
- [2] David Ha and Jürgen Schmidhuber. 2018. World Models. In *arXiv preprint arXiv:1803.10122*. <https://arxiv.org/abs/1803.10122>
- [3] Volodymyr Mnih, Antonio P. Badia, Mohammad Mirza, Alex Graves, Timothy Lillicrap, Tom Harley, David Silver, and Koray Kavukcuoglu. 2016. Asynchronous Methods for Deep Reinforcement Learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML 2016)*. JMLR.org, 1928–1937. <https://dl.acm.org/doi/10.5555/3045390.3045635>
- [4] Volodymyr Mnih and et al. 2015. Human-level control through deep reinforcement learning. *Nature* 518 (2015), 529–533. <https://doi.org/10.1038/nature14236>
- [5] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, David Hasselt, Matteo Hessel, Tristan Szegedy, and et al. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (2015), 529–533. <https://doi.org/10.1038/nature14236>
- [6] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. 2016. Prioritized experience replay. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [7] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017). <https://arxiv.org/abs/1707.06347>
- [8] Kenneth O. Stanley and Risto Miikkulainen. 2002. Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation* 10, 2 (2002), 99–127. <https://doi.org/10.1162/106365602320169811>
- [9] Hado van Hasselt, Arthur Guez, and David Silver. 2016. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 30. 2094–2100.
- [10] Ziyu Wang, Nando de Freitas, Marc G. Bellemare, Ian Osband, and Will Dabney. 2016. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*. 1995–2003.
- [11] Ronald J. Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning* 8, 3 (1992), 229–256. <https://link.springer.com/article/10.1007/BF00992696>